

Analisis Kluster pada Faktor-Faktor Numerik yang Mempengaruhi Harapan Hidup Global



Oleh :

Kelompok 5 ADD :

Jonathan Abimanyu Trisno	(5026231030)
James Melvin Chandra	(5026231076)
Yusuf Acala Sadurjaya Sri Krisna	(5026231089)
Davin Jonathan Tanus	(5026231131)

**DEPARTEMEN SISTEM INFORMASI
FAKULTAS TEKNOLOGI ELEKTRO DAN INFORMATIKA CERDAS
INSTITUT TEKNOLOGI SEPULUH NOPEMBER**

Ringkasan Perbaikan *Final Project*

ADD A

Semester Ganjil 2024-2025

Kelompok 9

Anggota (Nama dan NRP) :

1. Jonathan Abimanyu Trisno (5026231030)
2. James Melvin Chandra (5026231076)
3. Yusuf Acala Sadurjaya Sri Krisna (5026231089)
4. Davin Jonathan Tanus (5026231131)

No.	Perbaikan/Revisi	Halaman
1.		
2.		

DAFTAR ISI

Contents

DAFTAR ISI	2
BAB 1 PENDAHULUAN	3
1. Tema	3
2. Latar Belakang Masalah	3
3. Rumusan Masalah.....	3
4. Tujuan	3
5. Sumber Data	3
BAB 2 METODOLOGI PENELITIAN	4
2.1 Data Cleaning.....	4
2.2 Standardization.....	4
2.3 Principal Component Analysis (PCA)	4
2.3 K-Means Clustering	5
BAB 3 PEMBAHASAN	6
3.1 Pengenalan dan Persiapan Data	6
7	
3.2 Pembersihan Data.....	7
3.3 Standarisasi Fitur	11
3.3 K-Means Clustering pada Semua Fitur Data.....	14
3.4 Principal Component Analysis	19
3.6 Analisis Inferensial.....	33
3.7 Penarikan Kesimpulan.....	33
BAB 4 KESIMPULAN	34
DAFTAR PUSTAKA.....	36

BAB 1

PENDAHULUAN

1. Tema

Analisis Kluster pada Faktor-Faktor Numerik yang Mempengaruhi Harapan Hidup Global: Identifikasi Pola Berdasarkan Data Kesehatan dan Sosial

2. Latar Belakang Masalah

Harapan hidup merupakan indikator utama yang mencerminkan kualitas kesehatan, pendidikan, dan kesejahteraan sosial suatu populasi. Sebagai salah satu parameter utama yang digunakan oleh badan internasional seperti WHO dan PBB, harapan hidup dipengaruhi oleh berbagai faktor numerik seperti angka kematian, tingkat imunisasi, pengeluaran kesehatan, pendidikan, dan pendapatan.

Disparitas harapan hidup di seluruh dunia menggarisbawahi pentingnya analisis lebih mendalam terhadap faktor-faktor yang memengaruhinya. Melalui analisis kluster, pola-pola global dapat diidentifikasi untuk memberikan wawasan tentang pengelompokan data berdasarkan karakteristik numerik terkait kesehatan dan kesejahteraan.

Penelitian ini bertujuan untuk mengelompokkan data berdasarkan faktor numerik tanpa fokus pada wilayah geografis tertentu. Dengan memahami pola-pola tersebut, hasil analisis ini dapat menjadi dasar untuk merumuskan kebijakan kesehatan global yang lebih baik, khususnya dalam meningkatkan kualitas hidup masyarakat dunia.

3. Rumusan Masalah

- Apakah faktor-faktor numerik seperti angka kematian, imunisasi, pengeluaran kesehatan, tingkat pendidikan, dan pendapatan berpengaruh terhadap pola global harapan hidup?
- Bagaimana pola pengelompokan data berdasarkan faktor-faktor tersebut menggunakan algoritma K-Means?

4. Tujuan

- Mengidentifikasi faktor-faktor numerik yang memengaruhi pola global harapan hidup.
- Mengelompokkan data berdasarkan karakteristik numerik yang ada menggunakan algoritma K-Means.
- Memahami pola-pola yang terbentuk untuk memberikan wawasan tentang pengelolaan kebijakan kesehatan global secara lebih efektif.

5. Sumber Data

Data yang digunakan dalam studi ini diambil dari laman Kaggle dengan tautan sebagai berikut:

<https://www.kaggle.com/datasets/kumarajarshi/life-expectancy-who>.

Dataset ini mencakup indikator kesehatan, ekonomi, dan sosial dari tahun 2000 hingga 2015, yang disusun oleh WHO dan PBB.

BAB 2 METODOLOGI PENELITIAN

2.1 Data Cleaning

Data cleaning adalah proses untuk menangani data yang tidak lengkap atau tidak sesuai sehingga dapat digunakan dalam analisis lebih lanjut. Pada penelitian ini, dataset diperiksa menggunakan Python untuk mengidentifikasi nilai-nilai yang kosong (missing values).

Alih-alih menghapus baris atau kolom yang memiliki nilai kosong, kami menggunakan metode **K-Nearest Neighbors Imputer (KNNImputer)** untuk mengganti nilai kosong dengan nilai prediksi yang dihitung berdasarkan rata-rata data dengan kemiripan tertentu. Metode ini dipilih untuk menjaga integritas dataset sekaligus meminimalkan kehilangan informasi penting yang dapat memengaruhi hasil analisis.

2.2 Standardization

Untuk menghasilkan analisis *clustering* yang baik, standardisasi data adalah langkah yang penting untuk membuat setiap fitur data berada di skala yang sama. Metode standardisasi yang digunakan dalam penelitian ini adalah *Minmax Scaler*, yang menskala fitur pada *range* tertentu, biasanya dari 0 hingga 1, dimana minimum dari fitur dibuat sama dengan nol dan maximum dari fitur dibuat sama dengan satu. *Minmax Scaler* menskala data tanpa merubah bentuk distribusi dari data.

2.3 Principal Component Analysis (PCA)

Principal Component Analysis merupakan teknik *dimensional scaling* yang bertujuan untuk mengurangi dimensi suatu data tetapi tanpa mengurangi secara signifikan karakteristik dari data (Cahyadi, 2007). Cara *PCA* menyederhanakan data adalah dengan melakukan transformasi linear untuk membentuk sistem koordinat baru dengan variansi maksimum (Firlina dkk, 2015). Teknik ini mengubah himpunan banyak variabel yang saling berkorelasi menjadi himpunan variabel baru yang tidak saling berkorelasi. Tujuan utama dari *PCA* adalah untuk mengurangi *dataset* tetapi tetap mempertahankan sebanyak mungkin variansi. Langkah-langkah dari *PCA* adalah sebagai berikut (Geeksforgeeks, n.d.):

1. Standardisasi data

Data harus distandardisasi terlebih dahulu agar dapat melalui pengujian *PCA*. Dalam penelitian ini, metode standardisasi yang digunakan adalah *Minmax Scaler*.

2. Menghitung matriks kovarians

Kovarians merupakan perhitungan statistik yang mengukur variabilitas bersama dua atau lebih variabel, yang menggambarkan hubungan pergerakan antara seluruh variabel. Secara matematis, kovarians dapat dihitung dengan rumus berikut:

$$cov_{x,y} = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{N - 1}$$

Keterangan:

- Kovarians positif mengindikasikan bahwa jika x bertambah, maka y juga bertambah
- Kovarians negatif mengindikasikan bahwa jika x bertambah, maka y berkurang

- Kovarians nol mengindikasikan tidak adanya hubungan antara variabel
3. Menghitung nilai eigen dan vektor eigen dari matrix kovarians
Secara matematis, nilai eigen dan vektor eigen terkandung dalam persamaan berikut:

$$AX = \lambda X$$

Dimana λ adalah nilai eigen dari matrix A, dan X adalah vektor eigen dari matriks A untuk nilai eigen yang berkorespondensi.

2.3 K-Means Clustering

Clustering merupakan metode *unsupervised learning* dari *machine learning* untuk mengelompokkan data menjadi sejumlah kluster. Ada beberapa jenis metode *clustering*, dan metode yang umum digunakan adalah *K-Means clustering*. Langkah-langkah dari *K-Means clustering* adalah sebagai berikut (Wakhidah, 2010):

1. Menentukan jumlah kluster
2. Menentukan posisi awal *centroid* untuk semua kluster di titik acak pada koordinat Cartesius
3. Menghitung jarak dari setiap titik data ke seluruh *centroid*
4. Menentukan *centroid* mana yang paling dekat dengan suatu titik data, lalu menentukan *centroid* tersebut sebagai kelompok kluster untuk titik data tersebut.
5. Mengganti titik *centroid* pada setiap kluster menjadi titik tengah dari setiap titik-titik data di dalamnya
6. Lakukan iterasi pada langkah 3 hingga posisi *centroid* tidak berpindah-pindah lagi.

Untuk menentukan jumlah kluster, penelitian ini menggunakan metode *Elbow method*, dimana SSE (*Sum of Squared Error*) dari setiap jumlah kluster dihitung, dan titik dimana penurunan SSE dengan meningkatnya jumlah kluster tidak signifikan dipilih sebagai titik jumlah kluster yang dipilih.

BAB 3

PEMBAHASAN

Pada bab ini, kita akan membahas secara rinci analisis yang dilakukan dalam notebook `Banana Analysis.ipynb`. Notebook ini mengikuti beberapa langkah penting dalam analisis data sesuai dengan metodologi penelitian pada bab sebelumnya untuk mengoptimalkan kualitas panen pisang. Berikut adalah langkah-langkah dan penjelasan mendetail dari setiap tahap yang dilakukan dalam notebook tersebut.

3.1 Pengenalan dan Persiapan Data

Langkah penting pertama yang dilakukan dalam notebook adalah mengimpor beberapa library yang relevan untuk manipulasi data, visualisasi, pre-processing, dan clustering.

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

from sklearn.preprocessing import MinMaxScaler
```

pandas: library untuk bekerja dengan data tabular

matplotlib.pyplot: library untuk visualisasi data

numpy: library untuk komputasi numerik

sklearn.preprocessing: library untuk preprocessing data

Kemudian kita melakukan import file dataset yang akan kita uji

```
df = pd.read_csv("/Users/davin/Downloads/Life Expectancy Data.csv")
```

✓ 0.0s

```
df.head()
```

✓ 0.0s

- **df**: Variabel ini digunakan untuk menyimpan dataset yang dibaca dari file CSV.
- **pd.read_csv()**: Fungsi ini berasal dari pustaka pandas yang digunakan untuk membaca file CSV dan mengubahnya menjadi DataFrame. Pada kode ini, kita membaca file "Life Expectancy Data.csv" yang terletak pada path lokal "/Users/davin/Downloads/Life Expectancy Data.csv".
- **df.head()**: Fungsi ini digunakan untuk menampilkan 5 baris pertama dari DataFrame df. Hal ini bertujuan untuk melihat sekilas struktur data yang telah dibaca dan memastikan bahwa data telah dimuat dengan benar seperti bawah ini.

	Country	Year	Status	Life expectancy	Adult Mortality	infant deaths	Alcohol	percentage expenditure	Hepatitis B	Measles	...	Polio	Total expenditure	Diphtheria	HIV/AIDS	GDP	Population	thinness 1-19 years	thinness 5-9 years	Income composition of resources	Schooling
0	Afghanistan	2015	Developing	65.0	263.0	62	0.01	71.279624	65.0	1154	...	6.0	8.16	65.0	0.1	584.259210	33736494.0	17.2	17.3	0.479	10.1
1	Afghanistan	2014	Developing	59.9	271.0	64	0.01	73.523582	62.0	492	...	58.0	8.18	62.0	0.1	612.696514	327582.0	17.5	17.5	0.476	10.0
2	Afghanistan	2013	Developing	59.9	268.0	66	0.01	73.219243	64.0	430	...	62.0	8.13	64.0	0.1	631.744976	31731688.0	17.7	17.7	0.470	9.9
3	Afghanistan	2012	Developing	59.5	272.0	69	0.01	78.184215	67.0	2787	...	67.0	8.52	67.0	0.1	669.959000	3096958.0	17.9	18.0	0.463	9.8
4	Afghanistan	2011	Developing	59.2	275.0	71	0.01	7.097109	68.0	3013	...	68.0	7.87	68.0	0.1	63.537231	2978599.0	18.2	18.2	0.454	9.5

5 rows x 22 columns

3.2 Pembersihan Data

Pada tahap ini kita akan melakukan pengecekan tentang data yang bernilai null. output yang dihasilkan dari kode ini adalah jumlah sel yang null. Disini kita memiliki output 1289.

```
df.isnull().any(axis=1).sum()

1289
```

Kita juga bisa melihat kolom mana yang memiliki nilai null dan jumlahnya dengan kode dibawah ini. Kolom bernilai null memiliki nilai True dan kolom yang tidak memiliki nilai null bernilai False .

df.isnull().any(axis=0)		df.isnull().sum()	
Country	False	Country	0
Year	False	Year	0
Status	False	Status	0
Life expectancy	True	Life expectancy	10
Adult Mortality	True	Adult Mortality	10
infant deaths	False	infant deaths	0
Alcohol	True	Alcohol	194
percentage expenditure	False	percentage expenditure	0
Hepatitis B	True	Hepatitis B	553
Measles	False	Measles	0
BMI	True	BMI	34
under-five deaths	False	under-five deaths	0
Polio	True	Polio	19
Total expenditure	True	Total expenditure	226
Diphtheria	True	Diphtheria	19
HIV/AIDS	False	HIV/AIDS	0
GDP	True	GDP	448
Population	True	Population	652
thinness 1-19 years	True	thinness 1-19 years	34
thinness 5-9 years	True	thinness 5-9 years	34
Income composition of resources	True	Income composition of resources	167
Schooling	True	Schooling	163
dtype: bool		dtype: int64	

Di sini dapat dilihat bahwa beberapa kolom memiliki nilai null, dan jumlahnya juga tidak sedikit, karena itu langkah selanjutnya adalah melakukan imputasi untuk menggantikan nilai yang hilang tersebut. Salah satu metode imputasi yang digunakan dalam penelitian ini adalah KNN Imputer (K-Nearest Neighbors Imputer). Imputasi dengan KNN bekerja dengan cara menggantikan nilai yang hilang pada suatu sel dengan rata-rata nilai dari K-Nearest Neighbors berdasarkan kesamaan nilai dari atribut lainnya.

```
import pandas as pd
from sklearn.impute import KNNImputer
numeric_cols = df.select_dtypes(include=['float64', 'int64']).columns
imputer = KNNImputer(n_neighbors=5)
df[numeric_cols] = imputer.fit_transform(df[numeric_cols])
print(df.isnull().sum())
```

```
Country      0
Year         0
Status       0
Life expectancy  0
Adult Mortality  0
infant deaths  0
Alcohol      0
percentage expenditure  0
Hepatitis B  0
Measles      0
BMI          0
under-five deaths  0
Polio        0
Total expenditure  0
Diphtheria   0
HIV/AIDS    0
GDP          0
Population   0
thinness 1-19 years  0
thinness 5-9 years  0
Income composition of resources  0
Schooling    0
dtype: int64
```

Pada langkah ini, kita hanya memfokuskan pada kolom-kolom numerik dalam dataset, yaitu kolom yang bertipe data int64 dan float64, yang sering kali berisi nilai numerik yang relevan untuk analisis lebih lanjut. KNNImputer diterapkan dengan parameter `n_neighbors=5`, yang berarti algoritma ini akan mencari 5 tetangga terdekat dari setiap baris untuk mengimputasi nilai yang hilang. Setelah proses imputasi selesai, kita memeriksa kembali apakah masih ada nilai null dalam dataset.

```
df.isnull().any(axis=0)
```

```
Country      False
Year         False
Status       False
Life expectancy  False
Adult Mortality  False
infant deaths  False
Alcohol      False
percentage expenditure  False
Hepatitis B  False
Measles      False
BMI          False
under-five deaths  False
Polio        False
Total expenditure  False
Diphtheria   False
HIV/AIDS    False
GDP          False
Population   False
thinness 1-19 years  False
thinness 5-9 years  False
Income composition of resources  False
Schooling    False
dtype: bool
```

Pada blok kode selanjutnya kita menggunakan `df.shape` untuk menentukan dimensi dari dataset kita dan didapati bahwa data tersebut berdimensi (2938, 22)

```
df.shape

(2938, 22)
```

Selanjutnya, kita ingin membuat dataframe baru yang hanya berisi kolom-kolom numerik, yaitu data yang relevan untuk analisis kita. Untuk itu, kita akan menghapus kolom **Country**, **Year**, dan **Status**, karena kolom-kolom ini tidak memiliki hubungan langsung dengan analisis faktor-faktor yang mempengaruhi harapan hidup berdasarkan data numerik. Setelah dataframe baru dibuat, kita akan menampilkan 5 baris pertama menggunakan `.head()`. Dengan ini, kita bisa memastikan bahwa kolom **Country**, **Year**, dan **Status** telah berhasil dihapus dari dataframe.

```
df_features = df.drop(['Country', 'Year', 'Status'], axis=1)
df_features.head()
```

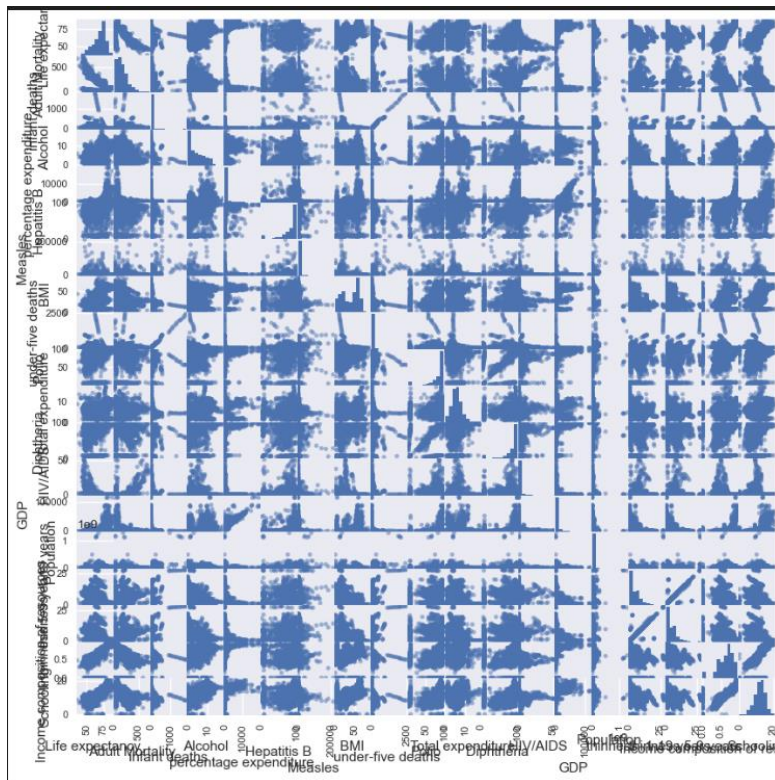
	Life expectancy	Adult Mortality	infant deaths	Alcohol	percentage expenditure	Hepatitis B	Measles	BMI	under-five deaths	Polio	Total expenditure	Diphtheria	HIV/AIDS	GDP	Population	thinness 1-19 years	thinness 5-9 years	Income composition of resources	Schooling
0	65.0	263.0	62.0	0.01	71.279624	65.0	1154.0	19.1	83.0	6.0	8.16	65.0	0.1	584.259210	33736494.0	17.2	17.3	0.479	10.1
1	59.9	271.0	64.0	0.01	73.523582	62.0	492.0	18.6	86.0	58.0	8.18	62.0	0.1	612.696514	327582.0	17.5	17.5	0.476	10.0
2	59.9	268.0	66.0	0.01	73.219243	64.0	430.0	18.1	89.0	62.0	8.13	64.0	0.1	631.744976	31731688.0	17.7	17.7	0.470	9.9
3	59.5	272.0	69.0	0.01	78.184215	67.0	2787.0	17.6	93.0	67.0	8.52	67.0	0.1	669.959000	3696958.0	17.9	18.0	0.463	9.8
4	59.2	275.0	71.0	0.01	7.097109	68.0	3013.0	17.2	97.0	68.0	7.87	68.0	0.1	63.537231	2978599.0	18.2	18.2	0.454	9.5

Untuk memvisualisasikan hubungan antar variabel numerik dalam dataset, kita dapat menggunakan scatter matrix plot. Scatter matrix plot ini akan menampilkan scatter plot untuk setiap pasangan kolom numerik dalam dataset.

Dalam kode ini, pertama-tama kita menggunakan `plt.style.use("seaborn")` untuk menerapkan gaya visual dari seaborn pada plot kita.

Kemudian, dengan menggunakan `pd.plotting.scatter_matrix(df_features, figsize=(10, 10))`, kita membuat scatter matrix plot untuk semua kolom numerik dalam DataFrame `df_features`. Parameter `figsize=(10, 10)` digunakan untuk mengatur ukuran plot agar sesuai dengan tampilan yang diinginkan. Berikut adalah kode yang digunakan untuk membuat plot dan contoh output yang dihasilkan.

```
plt.style.use("seaborn")
pd.plotting.scatter_matrix(df_features, figsize=(10, 10))
plt.show()
```



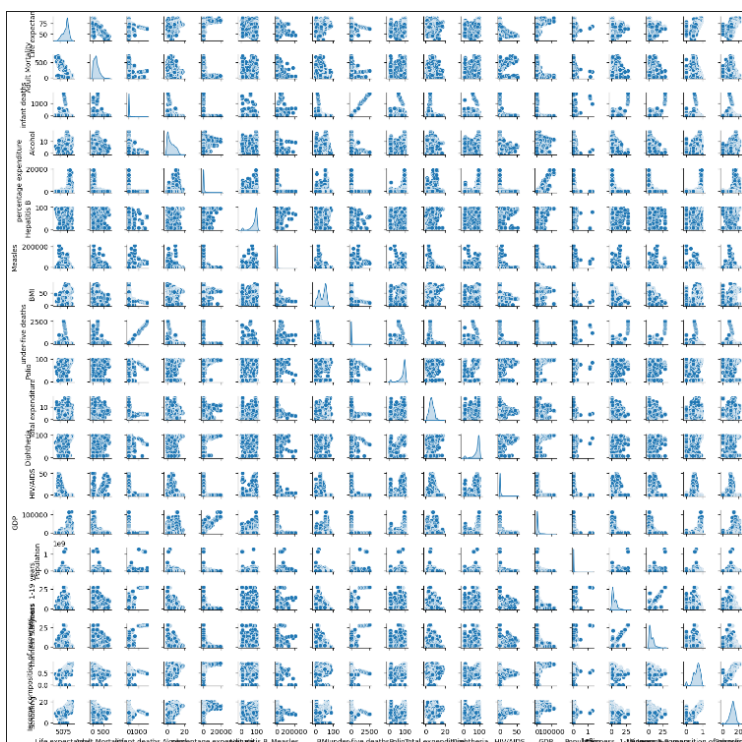
Dapat dilihat pada visualisasi di atas, scatter matrix plot yang dihasilkan memiliki ukuran dan detail yang cukup besar, sehingga sulit untuk dibaca dengan jelas. Untuk itu, kita bisa menggunakan pairplot sebagai alternatif yang lebih mudah dibaca dan lebih informatif dalam menampilkan hubungan antar variabel numerik.

```
import seaborn as sns
import matplotlib.pyplot as plt

# Tentukan ukuran figure
plt.figure(figsize=(5, 5))

# Visualisasikan pairplot
sns.pairplot(df_features, diag_kind='kde', height=0.8)
plt.show()
```

✓ 1m 26.2s



3.3 Standarisasi Fitur

Untuk membuat setiap fitur pada data berada pada skala yang sama, dilakukan proses standarisasi menggunakan **MinMaxScaler**. Standarisasi ini penting agar tidak ada fitur yang mendominasi analisis karena memiliki rentang nilai yang lebih besar.

Dengan menggunakan kode **MinMaxScaler().fit_transform(df_features)**, kita mengubah data numerik pada **df_features** sehingga semua nilai berada dalam rentang 0 hingga 1. Hasil dari proses ini disimpan dalam variabel **X**, dan kemudian dimasukkan ke dalam sebuah DataFrame baru yang bernama **df_minmax**, dengan kolom yang sama seperti pada **df_features**.

```
[16] X = MinMaxScaler().fit_transform(df_features)
✓ 0.0s

[17] df_minmax = pd.DataFrame(data = X, columns = df_features.columns.tolist())
✓ 0.0s
```

	Life expectancy	Adult Mortality	infant deaths	Alcohol	percentage expenditure	Hepatitis B	Measles	BMI	under-five deaths	Polio	Total expenditure	Diphtheria	HIV/AIDS	GDP	Population	thinness 1-19 years	thinness 5-9 years	Income composition of resources	Schooling
0	0.544592	0.362881	0.034444	0.000000	0.003659	0.653061	0.005439	0.209733	0.0332	0.031250	0.452118	0.649485	0.000000	0.004889	0.026074	0.619565	0.603509	0.505274	0.487923
1	0.447818	0.373961	0.035556	0.000000	0.003774	0.622449	0.002319	0.203940	0.0344	0.572917	0.453279	0.618557	0.000000	0.005127	0.000253	0.630435	0.610526	0.502110	0.483092
2	0.447818	0.369806	0.036667	0.000000	0.003759	0.642857	0.002027	0.198146	0.0356	0.614583	0.450377	0.639175	0.000000	0.005287	0.024525	0.637681	0.617544	0.495781	0.478261
3	0.440228	0.375346	0.038333	0.000000	0.004014	0.673469	0.013135	0.192352	0.0372	0.666667	0.473012	0.670103	0.000000	0.005608	0.002857	0.644928	0.628070	0.488397	0.473430
4	0.434535	0.379501	0.039444	0.000000	0.000364	0.683673	0.014200	0.187717	0.0388	0.677083	0.435287	0.680412	0.000000	0.000519	0.002302	0.655797	0.635088	0.478903	0.458937
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
2933	0.151803	1.000000	0.015000	0.243561	0.000000	0.683673	0.000146	0.302433	0.0168	0.666667	0.392339	0.649485	0.663366	0.003799	0.009875	0.336957	0.326316	0.429325	0.444444
2934	0.155598	0.988920	0.014444	0.226764	0.000000	0.061224	0.004703	0.297798	0.0164	0.041667	0.356936	0.680412	0.724752	0.003790	0.009764	0.351449	0.343860	0.440928	0.458937
2935	0.161290	0.099723	0.013889	0.247480	0.000000	0.734694	0.001433	0.293163	0.0160	0.729167	0.357516	0.711340	0.786139	0.000467	0.000097	0.039855	0.042105	0.450422	0.483092
2936	0.170778	0.948753	0.013889	0.095745	0.000000	0.765306	0.002493	0.288528	0.0156	0.760417	0.336042	0.752577	0.831683	0.004589	0.009558	0.054348	0.056140	0.450422	0.473430
2937	0.184061	0.919668	0.013333	0.093505	0.000000	0.795918	0.006989	0.283893	0.0156	0.781250	0.390598	0.783505	0.859406	0.004579	0.009446	0.394928	0.389474	0.457806	0.473430

Pada langkah berikutnya, kita melakukan visualisasi dengan **scatter matrix** untuk melihat hubungan antar fitur numerik yang telah distandarisasi.

aaaaaaaaaaaa

Standarisasi Fitur

Untuk membuat setiap fitur pada data banana_quality.csv berada pada skala yang sama, akan dilakukan standarisasi pada data. Standarisasi ini penting untuk menghindari bias dalam analisis selanjutnya.

```
x = MinMaxScaler().fit_transform(df)

standardized_df = pd.DataFrame(data = x, columns = df.columns.tolist())

standardized_df.insert(0, 'Quality', data['Quality'].values, False)

standardized_df

pd.plotting.scatter_matrix(standardized_df.drop('Quality', axis=1), figsize=(10, 10))
plt.show()
```

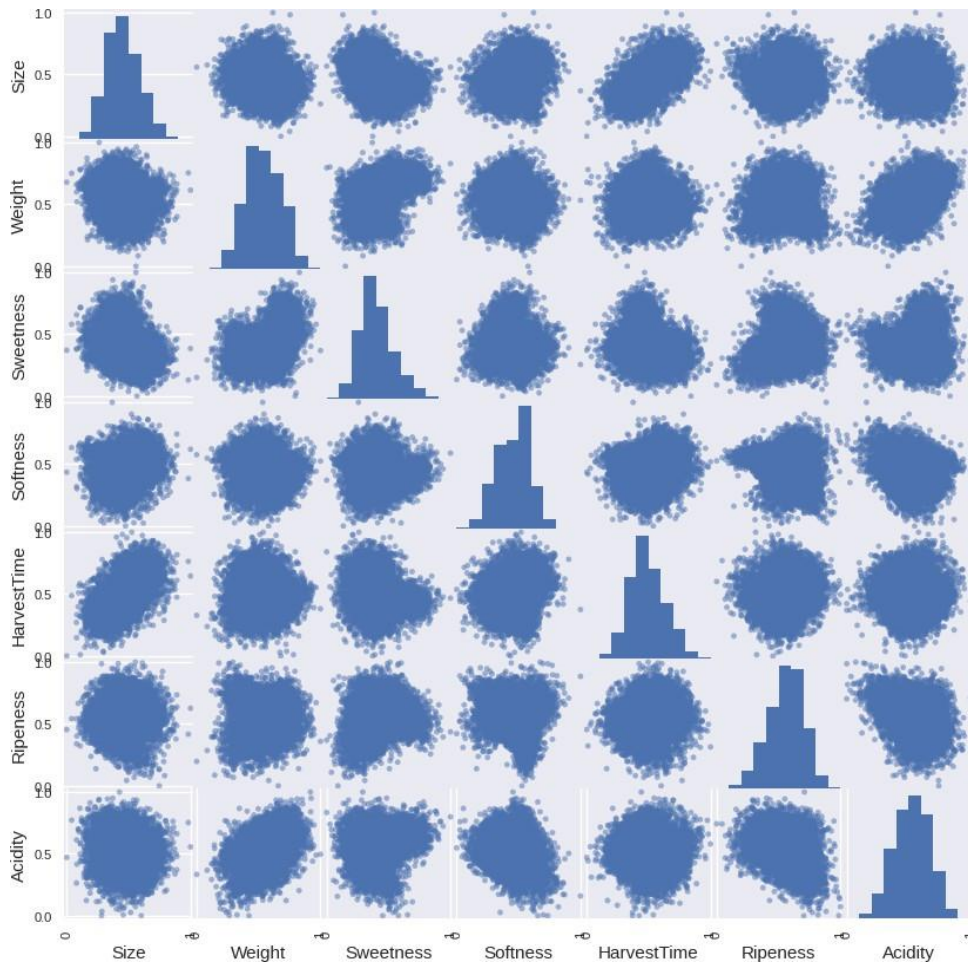
Standarisasi dilakukan dengan normalisasi menggunakan `MinMaxScaler().fit_transform(df)` dan memasukkannya ke variabel `x`. Kemudian, dibuat data baru bernama `standardized_df` dengan memasukkan nilai dari data banana_quality yang pertama untuk diambil column nya dan menggabungkannya dengan data `x`. Dalam `standardized_df` kita memasukkan kembali column “Quality” kedalam data bagian pertama, dan kemudian kita melakukan output untuk `standardized_df` sebagai berikut

	Quality	Size	Weight	Sweetness	Softness	HarvestTime	Ripeness	Acidity
0	Good	0.380309	0.626747	0.680712	0.360975	0.567312	0.671933	0.543416
1	Good	0.349951	0.628093	0.485275	0.293682	0.481689	0.646850	0.545720
2	Good	0.478460	0.699448	0.572694	0.283811	0.499358	0.716580	0.617337
3	Good	0.446465	0.705394	0.595677	0.374028	0.473461	0.633590	0.556625
4	Good	0.541672	0.687704	0.458841	0.378242	0.442847	0.579430	0.705908
...	...	...	...	...	...	...	...	...
7995	Bad	0.099172	0.645045	0.541670	0.652073	0.567537	0.495237	0.679413
7996	Bad	0.554154	0.434381	0.259196	0.490009	0.450586	0.348024	0.661191
7997	Bad	0.589947	0.456598	0.279220	0.521305	0.505481	0.380883	0.570661
7998	Bad	0.367350	0.396800	0.388309	0.597746	0.488151	0.261917	0.553153
7999	Bad	0.334225	0.446786	0.471829	0.556483	0.431943	0.396485	0.617870

8000 rows × 8 columns

Kemudian kita membentuk plot untuk `standardized_df` dengan ukuran `figsize(10, 10)` sebagai berikut.

```
pd.plotting.scatter_matrix(standardized_df.drop('Quality', axis=1), figsize=(10, 10))
plt.show()
```



3.4 K-Means Clustering pada Semua Fitur Data

Kemudian kita menginstall library kneed untuk mendukung kebutuhan clustering kita. Adapun detailnya sebagai berikut: KneeLocator dari library kneed, KMeans dari library sklearn.cluster dan silhouette_score dari library sklearn.metrics yang akan berfungsi dalam penghitungan K-Means Clustering dan nilai silhouette.

```
[ ] !pip install kneed
    from kneed import KneeLocator
    from sklearn.cluster import KMeans
    from sklearn.metrics import silhouette_score

Collecting kneed
  Downloading kneed-0.8.5-py3-none-any.whl (10 kB)
Requirement already satisfied: numpy>=1.14.2 in /usr/local/lib/python3.10/dist-packages (from kneed) (1.25.2)
Requirement already satisfied: scipy>=1.0.0 in /usr/local/lib/python3.10/dist-packages (from kneed) (1.11.4)
Installing collected packages: kneed
Successfully installed kneed-0.8.5

[ ] kmeans_kwargs = {
    "init": "random",
    "n_init": 10,
    "max_iter": 300,
    "random_state": 42
}

sse = []
for k in range(1,21):
    kmeans = KMeans(n_clusters=k, **kmeans_kwargs)
    kmeans.fit(x)
    sse.append(kmeans.inertia_)
```

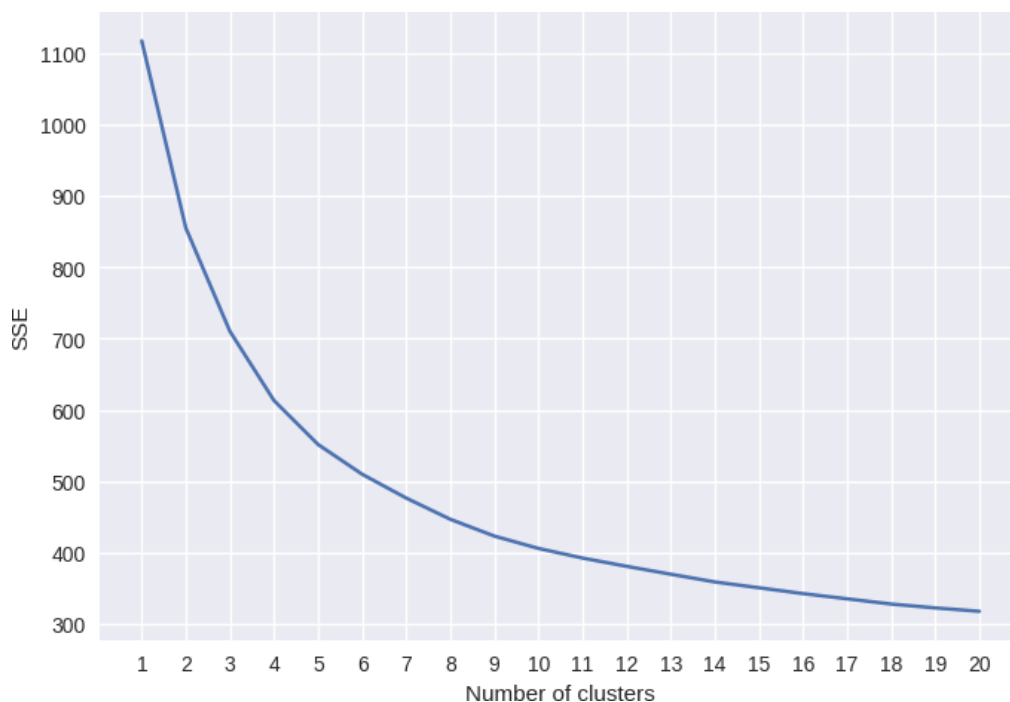
Kemudian kita membuat dictionary `kmeans_kwarg` dengan nilai:

- Memilih centroid awal secara acak `"init": "random"`
- Menentukan inisialisasi centroid sebanyak 10 kali `"n_init": 10`
- Menentukan maksimal jumlah iterasi sebanyak 300 kali `"max_iter": 300`
- Nilai seed untuk generator angka acak secara konsisten = 42 `"random_state": 42`

Selanjutnya yaitu menentukan nilai SSE (sum of squared error) dengan menentukan range k 1-21 dengan KMeans yang berisikan `kmeans_kwarg` yang telah diinisiasi sebelumnya.

Kemudian kita membuat plot dari range 1-21 dengan memasukkan jumlah cluster pada sumbu X dan SSE pada sumbu Y dan menampilkan outputnya sebagai berikut .

```
plt.plot(range(1,21), sse)
plt.xticks(range(1,21))
plt.xlabel("Number of clusters")
plt.ylabel("SSE")
plt.show()
```



Selanjutnya digunakan fungsi `KneeLocator` untuk menemukan titik siku dalam sebuah kurva. Fungsi tersebut mengambil rentang nilai (dalam hal ini dari 1 hingga 20), sum squared error (sse), jenis kurva (dalam hal ini "convex"), dan arah (dalam hal ini "decreasing") sebagai parameter. Kemudian, atribut `elbow` dari objek `k1` digunakan untuk mengakses titik siku yang ditemukan. Titik siku ini sering digunakan dalam analisis data untuk menentukan titik perubahan yang signifikan dalam sebuah kurva.

```

✓ 0s [17] # Find the elbow point
      k1 = KneLocator(range(1,21), sse, curve="convex", direction="decreasing")

      k1.elbow
⇨ 5

```

Pada cell ini dilakukan iterasi untuk mencoba berbagai jumlah klaster, dari 2 hingga 20. Setiap iterasi melibatkan pembuatan model K-Means dengan jumlah klaster yang sedang diuji. Model K-Means dilatih dengan data yang diberikan. Setelah pelatihan, nilai koefisien silhouette dihitung untuk mengevaluasi seberapa baik clustering telah dilakukan. Nilai koefisien silhouette untuk setiap jumlah klaster disimpan dalam sebuah array.

```

✓ 54s [18] silhouette_coef = []

      for i in range(2,21):
          kmeans = KMeans(n_clusters=i, **kmeans_kwargs)
          kmeans.fit(x)
          score = silhouette_score(x, kmeans.labels_)
          silhouette_coef.append(score)

```

Kode di bawah ini menghasilkan plot yang menunjukkan bagaimana nilai koefisien silhouette berubah dengan jumlah klaster yang berbeda, dari 2 hingga 20. Berikut adalah penjelasan singkatnya:

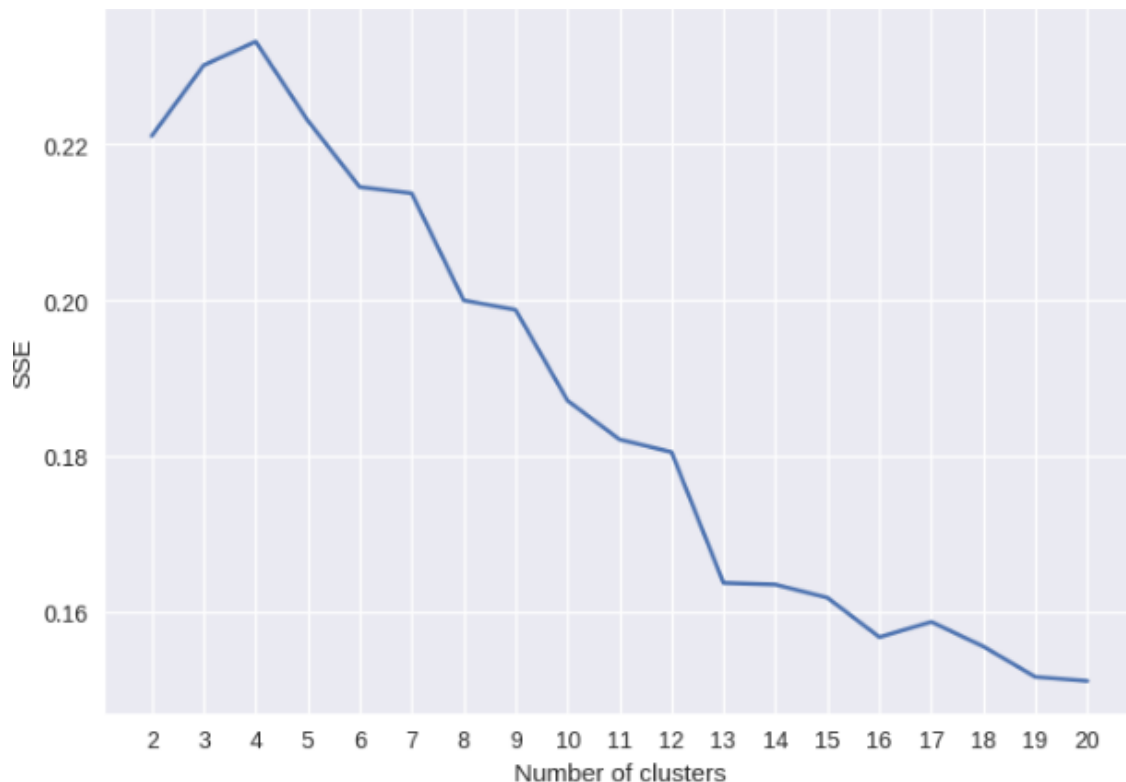
- `plt.plot(range(2,21), silhouette_coef)`: Ini membuat plot garis menggunakan nilai koefisien silhouette (`silhouette_coef`) sebagai nilai y, dengan jumlah klaster (2 hingga 20) sebagai nilai x.
- `plt.xticks(range(2,21))`: Ini menetapkan label sumbu x untuk menampilkan nilai dari 2 hingga 20 (jumlah cluster yang sedang diuji).
- `plt.xlabel("Number of clusters")`: Ini memberi label sumbu x sebagai "Number of clusters" (Jumlah klaster).
- `plt.ylabel("SSE")`: Ini memberi label sumbu y sebagai "SSE" (Sum of Squared Errors) meskipun seharusnya labelnya adalah "Silhouette Coefficient" berdasarkan konteks sebelumnya.
- `plt.show()`: Ini menampilkan plot.

```

✓ 0s [19] plt.plot(range(2,21), silhouette_coef)
      plt.xticks(range(2,21))
      plt.xlabel("Number of clusters")
      plt.ylabel("SSE")
      plt.show()

```

Plot ini membantu kita memvisualisasikan bagaimana jumlah kluster yang berbeda mempengaruhi nilai koefisien silhouette, yang membantu dalam menentukan jumlah cluster yang optimal.



Penentuan jumlah kluster: Berdasarkan analisis *elbow method* dan *silhouette coefficient*, jumlah kluster terbaik adalah 4 karena memiliki nilai *silhouette coefficient* yang tinggi dan termasuk cekungan pada grafik *elbow method*.

Selanjutnya, algoritma K-Means akan diterapkan pada dataset. Kode di bawah membuat objek model KMeans dengan parameter-parameter berikut:

- `init="random"`: Inisialisasi cluster secara acak. Ini berarti titik awal untuk pusat kluster akan dipilih secara acak dari data.
- `n_clusters=4`: Menentukan jumlah cluster yang ingin dibentuk. Dalam hal ini, jumlah kluster yang diinginkan adalah 4.
- `n_init=10`: Menentukan jumlah kali inisialisasi yang berbeda akan dilakukan. KMeans akan memulai dengan inisialisasi yang berbeda dan memilih solusi terbaik berdasarkan Sum of Squared Errors (SSE).
- `max_iter=300`: Menentukan jumlah iterasi maksimum yang akan dilakukan oleh algoritma pada setiap inisialisasi. Jika konvergensi belum tercapai setelah jumlah iterasi ini, algoritme akan berhenti.
- `random_state=42`: Menetapkan state untuk inisialisasi acak, sehingga hasilnya dapat direproduksi. Dalam hal ini, state yang ditetapkan adalah 42.

```

✓ [20] kmeans = KMeans(
0s      init="random",
      n_clusters = 4,
      n_init = 10,
      max_iter = 300,
      random_state = 42
  )

```

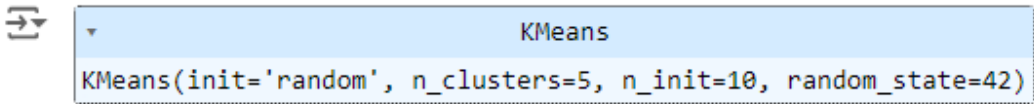
Objek kmeans yang dibuat dengan parameter-parameter ini kemudian dapat digunakan untuk melatih model KMeans dengan data yang diberikan.

Kode ini melatih model KMeans yang sudah dibuat (kmeans) menggunakan data yang diberikan (x). Proses pelatihan ini akan menempatkan titik data ke dalam kluster yang sesuai dan menghitung posisi pusat kluster (centroid) berdasarkan data yang diberikan.

```

✓ [21] kmeans.fit(x)
0s

```



Setelah proses pelatihan selesai, model KMeans akan siap digunakan untuk memprediksi kluster mana yang dimiliki oleh titik data baru atau untuk melakukan berbagai analisis tergantung pada kebutuhan spesifik.

Kode ini mengembalikan nilai inersia dari model KMeans setelah pelatihan. Inersia adalah ukuran dari seberapa jauh titik data dalam suatu kluster dari pusat klasternya. Semakin kecil nilai inersia, semakin padat kluster tersebut.

```

✓ [22] # The lowest SSE value
0s      kmeans.inertia_

551.7235612873736

```

Kode di bawah mengembalikan koordinat pusat (centroid) dari setiap kluster yang dihasilkan oleh model KMeans setelah pelatihan. Pusat kluster merupakan representasi titik pusat dari kelompok data dalam kluster tersebut.

```
✓ [23] # The centroids' position
0s kmeans.cluster_centers_

⇒ array([[0.40093228, 0.7106249 , 0.57907425, 0.39340735, 0.44623612,
          0.59317765, 0.64735253],
         [0.63700253, 0.55735264, 0.28433506, 0.52966766, 0.69738015,
          0.51151114, 0.55693087],
         [0.53896424, 0.42579251, 0.38678041, 0.52772069, 0.59700183,
          0.69301712, 0.34595953],
         [0.4185784 , 0.45906069, 0.33971047, 0.55632958, 0.43386221,
          0.3885316 , 0.60724543],
         [0.37450021, 0.52089871, 0.36562715, 0.33380837, 0.39656699,
          0.59200419, 0.46596877]])
```

Kode ini mengembalikan jumlah iterasi yang dibutuhkan oleh algoritma KMeans untuk konvergensi, yaitu jumlah iterasi yang diperlukan agar algoritma mencapai titik di mana pusat kluster tidak berubah lagi atau perubahan dalam pusat kluster sangat kecil.

```
✓ [24] # The number of iterations needed
0s kmeans.n_iter_

⇒ 21
```

Variabel `membership.dtype` ini akan mengembalikan tipe data dari array `membership` yang berisi label kluster untuk setiap titik data. Tipe data ini biasanya akan menjadi tipe data integer, karena label kluster adalah bilangan bulat yang menunjukkan kluster mana yang dimiliki oleh setiap titik data.

```
✓ [25] membership = kmeans.labels_
0s membership.dtype

⇒ dtype('int32')
```

3.5 Principal Component Analysis

PCA dalam analisis ini digunakan untuk mengurangi dimensi/fitur dalam dataset sambil mempertahankan sebanyak mungkin informasi (variabilitas) dari data. Teknik ini sering digunakan sebelum penerapan model machine learning untuk mengurangi kompleksitas data.

Hal pertama yang dilakukan pada PCA yaitu tentu saja mengimpor modul PCA dari library `scikit-learn` (`sklearn`).

```
✓ [26] from sklearn.decomposition import PCA
```

Selanjutnya, dibuatlah objek PCA dengan menggunakan fungsi `PCA()` yang diimpor sebelumnya dari modul `sklearn.decomposition`. Kemudian, objek PCA tersebut dilatih dengan

menggunakan metode `fit_transform()` pada data `x`, yang menghasilkan `x_pca`, yaitu representasi data `x` yang sudah direduksi dimensinya menggunakan PCA.

```
✓ [27] pca = PCA()
      x_pca = pca.fit_transform(x)
```

Variabel `pca.explained_variance_ratio_` adalah atribut dari objek PCA yang menyimpan proporsi dari varian data yang dijelaskan oleh setiap komponen utama (principal component) yang dihasilkan oleh PCA. Atribut ini mengembalikan larik satu dimensi yang berisi nilai-nilai antara 0 dan 1, di mana setiap nilai mewakili proporsi dari varian total yang dijelaskan oleh komponen utama yang bersangkutan.

```
✓ [28] pca.explained_variance_ratio_
0s
⇒ array([0.30758386, 0.20553604, 0.18123479, 0.12439765, 0.06838495,
        0.06136526, 0.05149744])
```

Variabel `x_pca.shape` mengembalikan bentuk (shape) dari array `x_pca`, yang mencerminkan jumlah baris dan kolom dari data yang sudah direduksi dimensinya menggunakan PCA. Jumlah baris akan sama dengan jumlah sampel dalam data awal `x`, sedangkan jumlah kolom akan sesuai dengan jumlah komponen utama (principal components) yang dihasilkan oleh PCA, yang mungkin lebih kecil dari jumlah fitur asli dalam data `x`.

```
✓ [29] x_pca.shape
0s
⇒ (8000, 7)
```

3.6 K-Means Clustering pada Fitur dengan Varians Tertinggi (Principal Component)

Kode di bawah berfungsi untuk membuat sebuah DataFrame Pandas (`pc`) dari data yang sudah direduksi dimensinya menggunakan PCA (`x_pca`). DataFrame ini akan mempertahankan empat komponen utama (principal components) pertama dari data `x_pca`, dan akan memberikan nama kolom 'pc1', 'pc2', 'pc3', dan 'pc4' untuk kedua komponen tersebut.

```
✓ [30] pc = pd.DataFrame(x_pca[:, :4], columns = ['pc1', 'pc2', 'pc3', 'pc4'])
0s
```

Selanjutnya, kita akan mentabulasikan indeks kluster yang telah dihasilkan dalam *list* membership pada tabel seperti di bawah, untuk *dataframe* `pc` dan data asli berturut-turut.

✓ 0s `pc['Cluster'] = membership`
`pc`

	pc1	pc2	pc3	pc4	Cluster
0	-0.174060	-0.153402	-0.187448	-0.089742	3
1	-0.163082	-0.137345	-0.070237	0.068821	3
2	-0.200515	-0.136593	-0.231587	0.052189	3
3	-0.195172	-0.094107	-0.157708	-0.050713	3
4	-0.174767	0.059918	-0.142524	0.072699	3
...	...	...	...	...	...
7995	-0.246346	0.113602	0.038913	-0.230146	3
7996	0.062192	0.257572	0.123624	0.110604	0
7997	0.131348	0.217325	0.070417	0.052043	0
7998	0.033008	0.238779	0.227378	-0.087826	0
7999	-0.087260	0.130549	0.163049	-0.092219	0

8000 rows × 5 columns

✓ 0s `df_cluster['Cluster'] = membership`
`df_cluster`

	Size	Weight	Sweetness	Softness	HarvestTime	Ripeness	Acidity	Cluster
0	0.380309	0.626747	0.680712	0.360975	0.567312	0.671933	0.543416	3
1	0.349951	0.628093	0.485275	0.293682	0.481689	0.646850	0.545720	3
2	0.478460	0.699448	0.572694	0.283811	0.499358	0.716580	0.617337	3
3	0.446465	0.705394	0.595677	0.374028	0.473461	0.633590	0.556625	3
4	0.541672	0.687704	0.458841	0.378242	0.442847	0.579430	0.705908	3
...	...	...	...	...	...	...	...	...
7995	0.099172	0.645045	0.541670	0.652073	0.567537	0.495237	0.679413	3
7996	0.554154	0.434381	0.259196	0.490009	0.450586	0.348024	0.661191	0
7997	0.589947	0.456598	0.279220	0.521305	0.505481	0.380883	0.570661	0
7998	0.367350	0.396800	0.388309	0.597746	0.488151	0.261917	0.553153	0
7999	0.334225	0.446786	0.471829	0.556483	0.431943	0.396485	0.617870	0

8000 rows × 8 columns

Untuk menganalisis karakteristik kuantitatif dari setiap pc pada setiap kluster, maka tabel dengan setiap kluster akan difilter

a. Kluster 0

```
df_cluster['Cluster'] = membership
df_cluster[df_cluster['Cluster'] == 0]
```

	Size	Weight	Sweetness	Softness	HarvestTime	Ripeness	Acidity	Cluster
4009	0.642600	0.543991	0.143092	0.427967	0.823640	0.414769	0.753517	0
4018	0.724024	0.568610	0.231537	0.599315	0.840555	0.316173	0.666590	0
4043	0.614365	0.498020	0.201314	0.502263	0.623209	0.420654	0.595122	0
4075	0.551240	0.601469	0.189764	0.460409	0.916984	0.311453	0.778887	0
4099	0.701150	0.733387	0.133767	0.567522	0.644265	0.376425	0.617918	0
...	...	...	...	...	...	...	...	...
7994	0.412617	0.602256	0.305186	0.570399	0.554370	0.427786	0.707336	0
7996	0.554154	0.434381	0.259196	0.490009	0.450586	0.348024	0.661191	0
7997	0.589947	0.456598	0.279220	0.521305	0.505481	0.380883	0.570661	0
7998	0.367350	0.396800	0.388309	0.597746	0.488151	0.261917	0.553153	0
7999	0.334225	0.446786	0.471829	0.556483	0.431943	0.396485	0.617870	0

1723 rows × 8 columns

Berikut merupakan rata-rata dari setiap fitur:

```
# Find the average of each features
[np.mean(df_cluster[df_cluster['Cluster'] == 0]['Size']),
 np.mean(df_cluster[df_cluster['Cluster'] == 0]['Weight']),
 np.mean(df_cluster[df_cluster['Cluster'] == 0]['Sweetness']),
 np.mean(df_cluster[df_cluster['Cluster'] == 0]['Softness']),
 np.mean(df_cluster[df_cluster['Cluster'] == 0]['HarvestTime']),
 np.mean(df_cluster[df_cluster['Cluster'] == 0]['Ripeness']),
 np.mean(df_cluster[df_cluster['Cluster'] == 0]['Acidity'])]
```

[0.44404134650405036,
0.4771207975831004,
0.32078221839772375,
0.5501185996021022,
0.4673596994437723,
0.37278944032417227,
0.6184506272732087]

Berikut merupakan rata-rata total dari seluruh fitur:

```
[138] # The total average
np.mean(df_cluster[df_cluster['Cluster'] == 0])
```

0.4063328411410162

b. Kluster 1

```

pc_cluster['cluster'] = membership
pc_cluster[pc_cluster['cluster'] == 1]

```

	pc1	pc2	pc3	pc4	cluster
2000	0.490312	-0.237822	-0.164290	0.210231	1
2012	0.144348	-0.223126	0.046607	-0.144157	1
2021	0.298347	-0.257750	-0.030433	0.014292	1
2024	0.186850	-0.140378	-0.120062	0.196424	1
2025	0.234505	-0.190215	-0.095061	0.105054	1
...	...	...	...	...	...
5995	0.302344	-0.048493	-0.081599	-0.190244	1
5996	0.208846	0.072179	-0.266148	0.143730	1
5997	0.223471	-0.019968	-0.002750	0.004159	1
5998	0.308327	-0.137192	-0.032533	-0.190554	1
5999	0.328913	0.049909	-0.212845	-0.112548	1

2218 rows × 5 columns

Berikut merupakan rata-rata dari setiap fitur:

```

# Find the average of each features
[np.mean(df_cluster[df_cluster['cluster'] == 1]['Size']),
 np.mean(df_cluster[df_cluster['cluster'] == 1]['Weight']),
 np.mean(df_cluster[df_cluster['cluster'] == 1]['Sweetness']),
 np.mean(df_cluster[df_cluster['cluster'] == 1]['Softness']),
 np.mean(df_cluster[df_cluster['cluster'] == 1]['HarvestTime']),
 np.mean(df_cluster[df_cluster['cluster'] == 1]['Ripeness']),
 np.mean(df_cluster[df_cluster['cluster'] == 1]['Acidity'])]

```

```

[0.5862121668463188,
 0.4707554869253232,
 0.3568588056800269,
 0.5336410755518054,
 0.6452187392715345,
 0.6361674070044507,
 0.4142076804626551]

```

Berikut merupakan rata-rata total dari seluruh fitur:

```

[141] # The total mean
np.mean(df_cluster[df_cluster['cluster'] == 1])

```

```

0.5803826702177644

```

c. Kluster 2

```

pc_cluster['Cluster'] = membership
pc_cluster[pc_cluster['Cluster'] == 2]

```

	pc1	pc2	pc3	pc4	Cluster
14	-0.129202	-0.343277	0.173465	0.335609	2
22	-0.215834	-0.146483	-0.026082	0.196003	2
57	-0.140473	-0.219296	0.059194	0.174418	2
70	-0.185566	-0.176891	-0.017518	0.099825	2
74	-0.158362	-0.188448	-0.027965	0.180686	2
...	...	...	...	...	...
7936	-0.093174	0.032708	0.056644	0.109347	2
7961	-0.078741	-0.077335	0.289986	-0.037042	2
7969	-0.222582	-0.087461	0.224718	-0.161526	2
7975	-0.112337	-0.167748	0.311555	-0.150208	2
7978	-0.020463	-0.082532	0.414555	-0.124668	2

2062 rows × 5 columns

Berikut merupakan rata-rata dari setiap fitur:

```

# Find the average of each features
[np.mean(df_cluster[df_cluster['Cluster'] == 2]['Size']),
 np.mean(df_cluster[df_cluster['Cluster'] == 2]['Weight']),
 np.mean(df_cluster[df_cluster['Cluster'] == 2]['Sweetness']),
 np.mean(df_cluster[df_cluster['Cluster'] == 2]['Softness']),
 np.mean(df_cluster[df_cluster['Cluster'] == 2]['HarvestTime']),
 np.mean(df_cluster[df_cluster['Cluster'] == 2]['Ripeness']),
 np.mean(df_cluster[df_cluster['Cluster'] == 2]['Acidity'])]

```

[0.3737706789743344,
0.5002048955483182,
0.3646012426540666,
0.3621267447637035,
0.3937334325892669,
0.5973961677669578,
0.45820726793379707]

Berikut merupakan rata-rata total dari seluruh fitur:

```

[142] # The total mean
np.mean(df_cluster[df_cluster['Cluster'] == 2])

```

0.6312550537788055

d. Kluster 3

0s  `pc_cluster['cluster'] = membership`
`pc_cluster[pc_cluster['cluster'] == 3]`



	pc1	pc2	pc3	pc4	cluster
0	-0.174060	-0.153402	-0.187448	-0.089742	3
1	-0.163082	-0.137345	-0.070237	0.068821	3
2	-0.200515	-0.136593	-0.231587	0.052189	3
3	-0.195172	-0.094107	-0.157708	-0.050713	3
4	-0.174767	0.059918	-0.142524	0.072699	3
...	...	...	...	...	...
7980	-0.324959	0.043723	0.087118	-0.018306	3
7983	-0.228709	0.251184	-0.114446	-0.083250	3
7991	-0.279628	-0.154190	0.166223	-0.221021	3
7992	-0.250095	0.133537	0.057607	-0.198187	3
7995	-0.246346	0.113602	0.038913	-0.230146	3

1997 rows × 5 columns

Berikut merupakan rata-rata dari setiap fitur:

 `# Find the average of each features`
`[np.mean(df_cluster[df_cluster['cluster'] == 3]['Size']),`
`np.mean(df_cluster[df_cluster['cluster'] == 3]['Weight']),`
`np.mean(df_cluster[df_cluster['cluster'] == 3]['Sweetness']),`
`np.mean(df_cluster[df_cluster['cluster'] == 3]['Softness']),`
`np.mean(df_cluster[df_cluster['cluster'] == 3]['HarvestTime']),`
`np.mean(df_cluster[df_cluster['cluster'] == 3]['Ripeness']),`
`np.mean(df_cluster[df_cluster['cluster'] == 3]['Acidity'])]`



[0.3986904996434714,
0.7071220661289896,
0.5741558912811012,
0.38899019490923586,
0.44396660531677884,
0.5949908735585496,
0.6429059478179671]

Berikut merupakan rata-rata total dari setiap fitur:

 `# The total mean`
`np.mean(df_cluster[df_cluster['cluster'] == 3])`



0.8438527598320117

Kita akan mencoba untuk menentukan jumlah dari setiap klaster. Berikut merupakan hasilnya

```
[ ] np.unique(membership)
↩ array([0, 1, 2, 3], dtype=int32)

[ ] unique, counts = np.unique(membership, return_counts=True)
    dict(zip(unique, counts))
↩ {0: 1723, 1: 2218, 2: 2062, 3: 1997}
```

Selanjutnya, kita akan mencoba memvisualisasikan kluster dalam bentuk *scatterplot*. Kode ini mendefinisikan beberapa warna dalam format heksadesimal untuk digunakan dalam representasi visual kluster. Kemudian, warna-warna tersebut digabungkan menjadi sebuah list `clust_colors`. Setiap kluster kemungkinan direpresentasikan dengan warna yang berbeda untuk membedakan antara satu kluster dengan kluster lainnya dalam visualisasi data.

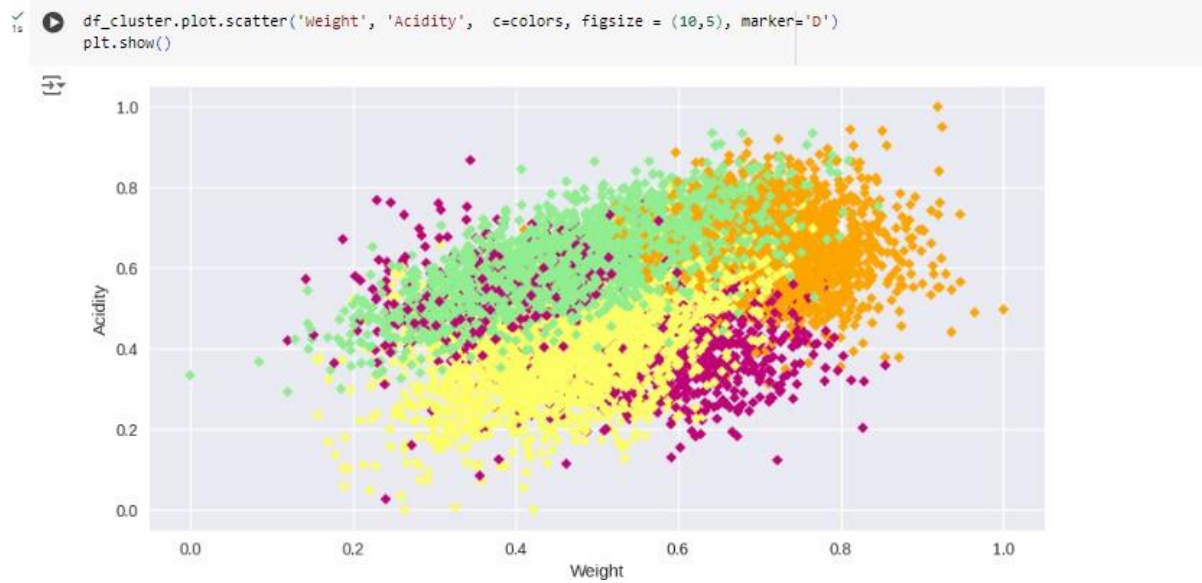
```
✓ [31] pale_green = ['#90ee90']
    yellow = ['#ffff66']
    magenta = ['#c20078']
    orange = ['#ffa500']
    blue = ['#0000ff']

    clust_colors = pale_green + yellow + magenta + orange + blue
```

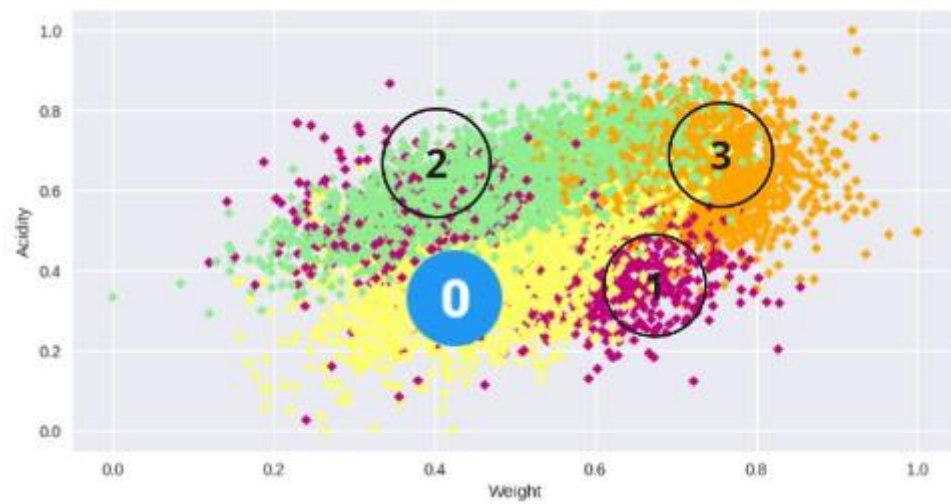
Kode ini menggunakan list comprehension untuk membuat list colors. Setiap elemen dalam list `membership` (yang berisi label kluster untuk setiap titik data) digunakan sebagai indeks untuk memilih warna yang sesuai dari list `clust_colors`. Akhirnya, list `colors` akan berisi warna yang sesuai untuk setiap titik data berdasarkan label kluster mereka.

```
✓ [32] colors = [clust_colors[m] for m in membership]
```

Selanjutnya, menggunakan fungsi `plot.scatter()` dari objek `DataFrame` `pc` untuk dibuatlah sebuah scatter plot yang memvisualisasikan dua komponen utama (`pc1` dan `pc2`) dari data yang sudah direduksi dimensinya menggunakan PCA. Warna dari setiap titik data di plot ditentukan oleh list `colors` yang sudah dibuat sebelumnya, yang menggambarkan kluster yang mereka miliki. Parameter `figsize` digunakan untuk menentukan ukuran dari plot yang dihasilkan, sedangkan parameter `marker` digunakan untuk menentukan jenis marker yang digunakan dalam plot (dalam hal ini, menggunakan marker berbentuk diamond 'D'). Dan akhirnya, `plt.show()` digunakan untuk menampilkan plot.



Pembagian kluster berdasarkan tabel kluster:



Langkah selanjutnya yaitu mencetak fitur-fitur PCA dari data.

```
[ ] pca_features = pd.DataFrame(x_pca, columns=['pc'+str(i) for i in range(1, x_pca.shape[1] + 1)])
pca_features
```



	pc1	pc2	pc3	pc4	pc5	pc6	pc7
0	-0.174060	-0.153402	-0.187448	-0.089742	-0.077714	-0.002130	0.117659
1	-0.163082	-0.137345	-0.070237	0.068821	0.018713	-0.018104	0.064737
2	-0.200515	-0.136593	-0.231587	0.052189	-0.035336	0.023855	-0.020439
3	-0.195172	-0.094107	-0.157708	-0.050713	0.000253	0.057281	0.006132
4	-0.174767	0.059918	-0.142524	0.072699	-0.027285	0.036256	-0.109944
...	...	...	...	...	...	...	...
7995	-0.246346	0.113602	0.038913	-0.230146	0.038447	-0.267201	0.161646
7996	0.062192	0.257572	0.123624	0.110604	-0.072309	0.038871	-0.057373
7997	0.131348	0.217325	0.070417	0.052043	-0.031145	0.066854	-0.039940
7998	0.033008	0.238779	0.227378	-0.087826	-0.063864	0.007176	0.123357
7999	-0.087260	0.130549	0.163049	-0.092219	-0.100993	-0.040932	0.062516

8000 rows x 7 columns

Menghitung variansi setiap kolom kecuali kolom Quality pada data yang dinormalisasi.

```
[ ] df_minmax_vars = standardized_df.drop('Quality', axis=1).var()
df_minmax_vars
```



Size	0.017892
Weight	0.020846
Sweetness	0.019444
Softness	0.018458
HarvestTime	0.020743
Ripeness	0.020765
Acidity	0.021507

dtype: float64

Melakukan normalisasi serta pengurutan pada variansi.

```
[ ] df_minmax_varsum = sum(df_minmax_vars)
np.sort(-np.array([a/df_minmax_varsum for a in df_minmax_vars]))
```



```
array([-0.15400319, -0.14926413, -0.14868947, -0.14853105, -0.13922524,
       -0.13217063, -0.12811629])
```

Kali ini, akan dilakukan proses clustering menggunakan k-means clustering pada principal components. Berikut proses clustering pada seluruh fitur PCA.

```
[ ] kmeans_pca_all = KMeans(
    init="random",
    n_clusters=4,
    n_init=10,
    max_iter=300,
    random_state=42
)
```

```
[ ] kmeans_pca_all.fit(pca_features)
```

```
↳ KMeans
KMeans(init='random', n_clusters=5, n_init=10, random_state=42)
```

```
[ ] # The lowest SSE value
kmeans.inertia_
```

```
↳ 551.7235612873736
```

```
[ ] # The centroids' position
kmeans.cluster_centers_
```

```
↳ array([[0.40093228, 0.7106249 , 0.57907425, 0.39340735, 0.44623612,
          0.59317765, 0.64735253],
        [0.63700253, 0.55735264, 0.28433506, 0.52966766, 0.69738015,
          0.51151114, 0.55693087],
        [0.53896424, 0.42579251, 0.38678041, 0.52772069, 0.59700183,
          0.69301712, 0.34595953],
        [0.4185784 , 0.45906069, 0.33971047, 0.55632958, 0.43386221,
          0.3885316 , 0.60724543],
        [0.37450021, 0.52089871, 0.36562715, 0.33380837, 0.39656699,
          0.59200419, 0.46596877]])
```

```
[ ] # The number of iterations needed
kmeans.n_iter_
```

```
↳ 21
```

Selanjutnya dilakukan clustering pada fitur PCA pertama (pc1) sebagai berikut.

```

kmeans_pca_first = KMeans(
    init="random",
    n_clusters=4,
    n_init=10,
    max_iter=300,
    random_state=42
)

```

```
[ ] kmeans_pca_first.fit(pca_features.pc1.values.reshape(-1,1))
```

```

KMeans
KMeans(init='random', n_clusters=5, n_init=10, random_state=42)

```

```
[ ] # The lowest SSE value
kmeans.inertia_
```

```
551.7235612873736
```

```
[ ] # The centroids' position
kmeans.cluster_centers_
```

```

array([[0.40093228, 0.7106249, 0.57907425, 0.39340735, 0.44623612,
        0.59317765, 0.64735253],
       [0.63700253, 0.55735264, 0.28433506, 0.52966766, 0.69738015,
        0.51151114, 0.55693087],
       [0.53896424, 0.42579251, 0.38678041, 0.52772069, 0.59700183,
        0.69301712, 0.34595953],
       [0.4185784, 0.45906069, 0.33971047, 0.55632958, 0.43386221,
        0.3885316, 0.60724543],
       [0.37450021, 0.52089871, 0.36562715, 0.33380837, 0.39656699,
        0.59200419, 0.46596877]])

```

```
[ ] # The number of iterations needed
kmeans.n_iter_
```

```
21
```

Clustering pada dua fitur pertama PCA (pc1, pc2).

```
[ ] kmeans_pca_first2 = KMeans(
    init="random",
    n_clusters=4,
    n_init=10,
    max_iter=300,
    random_state=42
)
```

```
[ ] kmeans_pca_first2.fit(pca_features[['pc1', 'pc2']])
```

```
↳ KMeans
KMeans(init='random', n_clusters=5, n_init=10, random_state=42)
```

```
[ ] # The lowest SSE value
kmeans.inertia_
```

```
↳ 551.7235612873736
```

```
[ ] # The centroids' position
kmeans.cluster_centers_
```

```
↳ array([[0.40093228, 0.7106249 , 0.57907425, 0.39340735, 0.44623612,
          0.59317765, 0.64735253],
        [0.63700253, 0.55735264, 0.28433506, 0.52966766, 0.69738015,
          0.51151114, 0.55693087],
        [0.53896424, 0.42579251, 0.38678041, 0.52772069, 0.59700183,
          0.69301712, 0.34595953],
        [0.4185784 , 0.45906069, 0.33971047, 0.55632958, 0.43386221,
          0.3885316 , 0.60724543],
        [0.37450021, 0.52089871, 0.36562715, 0.33380837, 0.39656699,
          0.59200419, 0.46596877]])
```

```
[ ] # The number of iterations needed
kmeans.n_iter_
```

```
↳ 21
```

Selanjutnya, kita akan melakukan k-means clustering pada tiga fitur PCA pertama, yaitu pc1, pc2, dan pc3.

```
[ ] kmeans_pca_first3 = KMeans(
    init="random",
    n_clusters=4,
    n_init=10,
    max_iter=300,
    random_state=42
)
```

```
[ ] kmeans_pca_first3.fit(pca_features[['pc1', 'pc2', 'pc3']])
```

```
↗ KMeans
KMeans(init='random', n_clusters=5, n_init=10, random_state=42)
```

```
[ ] # The lowest SSE value
    kmeans.inertia_
```

```
↗ 551.7235612873736
```

```
✓ [55] # The centroids' position
0s kmeans.cluster_centers_

↗ array([[0.40093228, 0.7106249 , 0.57907425, 0.39340735, 0.44623612,
          0.59317765, 0.64735253],
         [0.63700253, 0.55735264, 0.28433506, 0.52966766, 0.69738015,
          0.51151114, 0.55693087],
         [0.53896424, 0.42579251, 0.38678041, 0.52772069, 0.59700183,
          0.69301712, 0.34595953],
         [0.4185784 , 0.45906069, 0.33971047, 0.55632958, 0.43386221,
          0.3885316 , 0.60724543],
         [0.37450021, 0.52089871, 0.36562715, 0.33380837, 0.39656699,
          0.59200419, 0.46596877]])
```

```
✓ [56] # The number of iterations needed
0s kmeans.n_iter_
```

```
↗ 21
```

Langkah terakhir, kita hitung perbandingan dari data (yang telah dinormalisasi) dengan semua data, satu komponen pertama, dua komponen pertama, dan tiga komponen pertama dari komponen PCA data. Untuk ini, kita akan menggunakan fungsi `v_measure_score` dari `sklearn.metrics` yang kita impor terlebih dahulu

```
✓ [57] from sklearn.metrics import v_measure_score
0s
```

Perbandingan data (yang telah dinormalisasi) dengan semua komponen PCA.

```
✓ [58] # Between scaled data and all PCA components
0s v_measure_score(kmeans.labels_, kmeans_pca_all.labels_)
```

```
↗ 1.0
```

Perbandingan data (yang telah dinormalisasi) dengan komponen PCA pertama.

```
✓ [59] # Between scaled data and first one PCA component
0s v_measure_score(kmeans.labels_, kmeans_pca_first.labels_)
⇨ 0.41844637828505526
```

Perbandingan data (yang telah dinormalisasi) dengan dua komponen PCA pertama.

```
✓ [60] # Between scaled data and first two PCA components
0s v_measure_score(kmeans.labels_, kmeans_pca_first2.labels_)
⇨ 0.5959700096880778
```

Perbandingan data (yang telah dinormalisasi) dengan tiga komponen PCA pertama.

```
✓ [61] # Between scaled data and first three PCA components
0s v_measure_score(kmeans.labels_, kmeans_pca_first3.labels_)
⇨ 0.7041046267971919
```

3.7 Analisis Inferensial

Berdasarkan tabel kluster dan visualisasi, diperoleh kesimpulan sebagai berikut:

1. Kluster 0 merupakan metrik yang rata-rata fiturnya 0,4063, berjumlah 1723 data
2. Kluster 1 merupakan metrik yang rata-rata fiturnya 0,5804, berjumlah 2218 data
3. Kluster 2 merupakan metrik yang rata-rata fiturnya 0,6313, berjumlah 2062 data
4. Kluster 3 merupakan metrik yang rata-rata fiturnya 0,8439, berjumlah 1997 data

3.8 Penarikan Kesimpulan

Pada pisang, kita menginginkan seluruh metrik kuantitas (keasaman, kemanisan, dll) tidak terlalu tinggi dan tidak terlalu rendah. Jika dihitung dari skala 0 hingga 1, maka angka yang ideal adalah 0.4 hingga 0.6. Kluster yang memenuhi adalah kluster 0, kluster 1, dan kluster 2. Proporsi mereka adalah $6003/8000 = 75,04\%$.

Dengan demikian, kualitas hasil panen pisang di Indonesia saat ini adalah 75,04%.

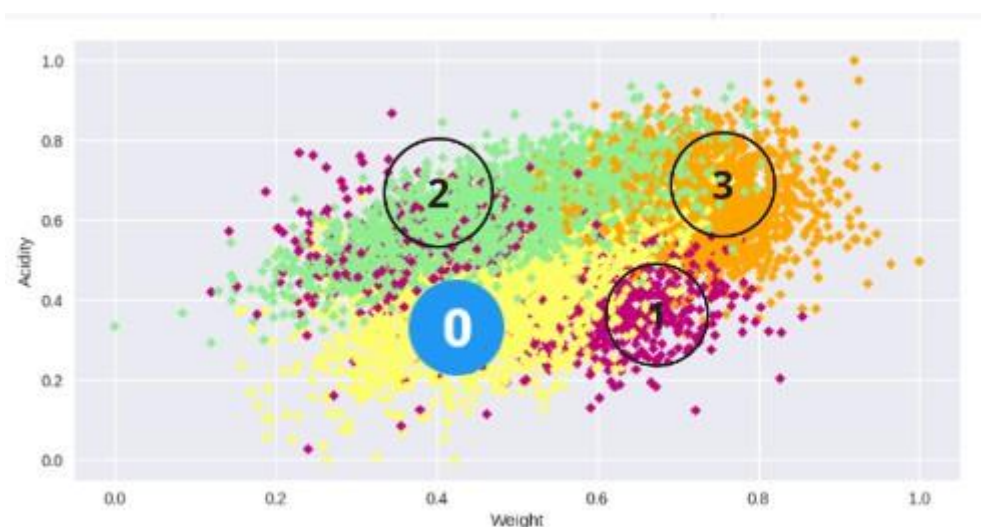
BAB 4

KESIMPULAN

Pertanian merupakan sektor yang berkontribusi besar terhadap Produk Domestik Bruto (PDB) Indonesia dan kesejahteraan penduduk. Pada tahun 2020, kontribusi pertanian terhadap PDB negara adalah 13,7% (Badan Pusat Statistik, 2023). Salah satu komoditas pertanian yang sangat penting adalah pisang, bahkan pada tahun 2014 produksi pisang menempati posisi pertama di antara komoditas pertanian yang lainnya. Dalam analisis dalam peningkatan produktivitas dan kesejahteraan petani terkhusus dalam optimalisasi kualitas panen pisang dimana menurut Standar Nasional Indonesia Pisang dikategorikan dalam 3 kelas berdasarkan mutu produk, terdiri dari kelas super, A dan B.

Dengan menggunakan metodologi penelitian yang terdiri dari *Data Cleaning* untuk menghapus data yang tidak memiliki nilai untuk menghindari gangguan ketika analisis, *standardization* dengan teknik *minmax scaler* dimana menggunakan nilai 0 sebagai nilai minimum dan 1 untuk nilai maksimum guna melakukan *scaling* data tanpa merubah bentuk distribusi dari data, *principal component analysis (PCA)* untuk mengurangi dimensi data tanpa menghilangkan karakteristik data dan terakhir dengan *K-Means Clustering* untuk menghitung jarak setiap titik data ke seluruh *centroid* teknik teknik tersebut digunakan untuk menghasilkan analisis yang *reliable* dalam optimalisasi kualitas panen pisang sebagai salah satu komoditas penting dalam pertanian.

Dalam pembahasan ini, hasil dari berbagai metode analisis yang diterapkan pada notebook "Banana Analysis.ipynb" menunjukkan beberapa temuan penting dalam upaya mengoptimalkan kualitas panen pisang. Pada tahap pembersihan data, diperoleh hasil bahwa tidak ada data yang hilang, dengan dimensi dataset sebesar (8000, 7). Standarisasi fitur menggunakan MinMaxScaler berhasil menormalkan data, memastikan setiap fitur berada pada skala yang sama, sehingga menghindari bias dalam analisis berikutnya. Pada dataset dilakukan PCA dan K-Means clustering yang menghasilkan *output* yaitu tabel kluster dan visualisasi *scatterplot* sebagai berikut:



Berdasarkan analisis, posisi kluster dimana kualitas pisang adalah terbaik, adalah di kluster nomor 0, 1 dan 2. Proporsi mereka adalah 75,04% dari keseluruhan, sehingga dapat disimpulkan bahwa kualitas hasil panen pisang di Indonesia saat ini adalah 75,04%.

DAFTAR PUSTAKA

- Sabarella dkk. (2023). *ANALISIS PDB SEKTOR PERTANIAN TAHUN 2023*. Kementerian Pertanian Indonesia.
- BPS. (2023). *Produk Domestik Bruto Indonesia Triwulanan 2019-2023*. Jakarta: Oktober 2023. Badan Pusat Statistik.
- Direktorat Jendral Hortikultura. (2015). *Statistik Produksi Hortikultura Tahun 2014*. Direktorat Jendral Hortikultura, Kementrian Pertanian. Jakarta: Indonesia.
- RumahRupute, B. (2007). *Pengembangan dan Pasca Panen Pisang (Musa sp)*. Balai Besar Pengkajian dan Pengembangan Teknologi Pertanian
- Mujiyo dkk. (2017). *POTENSI LAHAN UNTUK BUDIDAYA PISANG DI KECAMATAN JENAWI KARANGANYAR*. Caraka Tani: Journal of Sustainable Agriculture. 2017. 32(2), 142-148
- Cahyadi, Daniel. 2007. Ekstraksi dan Kemiripan. Universitas Indonesia.
- Wakhidah, N. (2010). Clustering menggunakan k-means algorithm. Jurnal Transformatika, 8(1), 33-39.